

Table of Contents

1. The Best List (TL;DR)	2
1.1 Which pick matches which scenario	2
2. What This App Needs to Run	3
2.1 Required environment variables	3
2.2 Background work and process model	4
3. DigitalOcean Deep Dive: Hosting the Web App	5
3.1 App Platform versus Droplets	5
3.2 Managed PostgreSQL and DOKS	5
4. DigitalOcean GPU Droplets: The Mining Angle	6
4.1 The honest mining economics	6
5. Cross-Provider Comparison	7
5.1 GPU providers versus DigitalOcean	7
6. Production Blueprint: Deploy on a DigitalOcean Droplet	8
6.1 Steps 1-4: create, harden, install, configure	8
6.2 Steps 5-7: build, supervise, serve over TLS	9
6.3 Steps 8-10: backups, monitoring, updates	9
7. Cost Scenarios and FAQ	10
7.1 Frequently asked questions	10

1. The Best List (TL;DR)

This guide answers the deployment and hosting questions for the InfraNexBT web platform and delivers a researched, price-verified DigitalOcean list as of September 2026. The short version: the application is a single Next.js 16 standalone server with a SQLite database, a small set of background workers, and outbound integrations to SSH hosts, the Bittensor chain, GitHub, and cloud providers. That profile fits a modest Virtual Private Server perfectly, and a DigitalOcean Droplet is the strongest overall fit because the same account also acts as a provisioning provider inside the platform itself. The table below ranks every option this guide covers; the chapters that follow justify each pick with verified pricing and trade-offs.

Pick	Choice	Verified price	Why
Best overall (web app)	DO Droplet, Basic 2 vCPU / 4 GB	\$24/mo (\$0.0357/hr)	Comfortable build + run headroom, full filesystem control, snapshots
Best budget	DO Droplet, 1 vCPU / 1 GB	\$6/mo	Runs the prebuilt standalone server fine; build off-board or add swap
Best zero-ops	DO App Platform, Basic web + worker	from \$10/mo (2 services)	GitHub-to-deploy with no server admin; forces DB migration off SQLite
Managed database	DO Managed PostgreSQL	from \$15/mo (HA \$30/mo)	Automatic backups and failover when the app outgrows SQLite
Cheapest DO GPU	NVIDIA RTX 4000 Ada (20 GB)	\$0.76/hr	Only DigitalOcean GPU whose hourly rate can plausibly pay for itself in mining trials
Balanced DO GPU	RTX 6000 Ada or L40S (48 GB)	\$1.57/hr	Serious inference or validation workloads in Toronto (TOR1)
High-end DO GPU	H100 80 GB (SXM)	from about \$3.39/hr; 8x node \$35.28/hr	AI training class hardware; rarely economic for subnet mining
Budget GPU alternative	RunPod RTX 4090 (24 GB)	\$0.34-\$0.69/hr	Cheapest integrated path for experiments; community cloud is spot-priced

Table 1: The ranked picks with verified September 2026 pricing.

\$24/mo
Recommended control-plane Droplet (2 vCPU / 4 GB)

\$0.76/hr
Cheapest DigitalOcean GPU (RTX 4000 Ada, TOR1)

\$15/mo
Managed PostgreSQL single node (1 vCPU / 1 GiB)

1.1 Which pick matches which scenario

Scenario	Deploy this	Estimated monthly
Just run the dashboard for a few users	1 vCPU / 1 GB Droplet + swap + snapshots	\$7-\$8
Production control plane	2 vCPU / 4 GB Droplet + nightly SQLite backups + Caddy TLS	\$29-\$34
Hands-off operation, no server admin	App Platform web + worker + Managed PostgreSQL	\$35-\$40
Trial GPU mining on DigitalOcean	RTX 4000 Ada GPU Droplet in TOR1, hourly, stop-loss checked	~\$554 if 24/7
Multi-provider miner fleet	\$24 Droplet control plane + RunPod/Vast GPU workers behind DevOps Engine	\$34 + metered GPU

Table 2: Scenario-to-pick mapping with realistic all-in monthly cost.

Two honest caveats belong up front. First, DigitalOcean GPU rates are competitive with mainstream clouds but are undercut by RunPod and Vast.ai for the consumer-class cards that dominate Bittensor mining, so keep the control plane and the mining fleet decisions separate. Second, the \$24 Droplet recommendation assumes the platform keeps its SQLite database; if you prefer managed PostgreSQL from day one, the same Droplet plus the \$15 database lands near \$39 per month before snapshots. Both paths are detailed in chapters 3 and 6.

2. What This App Needs to Run

Hosting advice grounded in the actual codebase beats generic Next.js checklists, so this chapter states the verified runtime profile of the InfraNexBT platform. The app builds to a standalone Next.js 16 server (React 19, Tailwind 4, shadcn/ui) and is started with Bun via `bun .next/standalone/server.js` on port 3000. Data lives in a single-file Prisma 6 SQLite database at `db/custom.db`, which means hosting requires a persistent filesystem and file-level backups rather than a stateless container. The dependency list includes `ssh2` (the platform opens real SSH sessions to provisioned hosts), the Polkadot API (live Bittensor chain reads), and `z-ai-web-dev-sdk` (AI analysis features), so the server needs unrestricted outbound HTTPS plus outbound SSH, and accurate clock sync because the daemon channel signs requests with time-bounded HMAC signatures.

2.1 Required environment variables

Variable	Required	Purpose
DATABASE_URL	Yes	Prisma connection; file:/opt/infranex/db/custom.db on a Droplet
APP_SESSION_SECRET	Yes	Signs session cookies; rotating it logs out all users
DEVOPS_SECRET	Yes	HMAC key for the daemon command/telemetry channel; lost if regenerating it invalidates paired miners
INFRANEX_REPO_ROOT	Production	Absolute project path used by the standalone server start command
RUNPOD_API_KEY	Optional	Fallback RunPod credential; provider keys are normally stored in the database via the UI
INFRANEX_PLATFORM_URL	Optional	Self-referential public URL used by the daemon bridge

Table 3: Environment variables referenced by the codebase (verified via source scan).

2.2 Background work and process model

In the current development setup seven background workers cycle inside the dev server, and maintenance scripts such as the chain reconcile job run under Bun. On a production Droplet the web server becomes one systemd unit, and recurring jobs become systemd timers or cron entries calling bun scripts/.ts. Nothing in the codebase requires a message broker; the daemon command queue lives in the application database and is drained by miners polling /api/daemon/commands every 60 seconds. That design keeps a single small server viable and is the main reason Kubernetes is not recommended here. RAM is the one real constraint: the production build (next build) needs roughly 2 GB, so either pick the 4 GB tier or build the artifact elsewhere and copy .next/standalone to a 1 GB box.

One warning about the repository root: the checked-in docker-compose.yml is a stale template from the original scaffold. It references a Python backend on port 8000, a Supabase Postgres image, and a Redis worker that do not exist in this application. Do not deploy from it, and do not let an automation tool discover it and assume it is the intended stack. The supported topology is the standalone server plus systemd, which chapter 6 walks through step by step.

3. DigitalOcean Deep Dive: Hosting the Web App

DigitalOcean offers four plausible homes for this application: classic Droplets, the App Platform PaaS, Managed PostgreSQL for the data layer, and Kubernetes (DOKS). Verified September 2026 pricing makes the comparison unusually one-sided for a workload like this one. Droplets bill per second with a 60-second minimum, usage is capped at 672 hours per month, and each plan includes a monthly outbound transfer allowance, so a control plane that idles at low CPU costs exactly its plan price.

Droplet plan (Basic)	vCPU / RAM / Disk	Price	Fit for this app
Entry	1 vCPU / 512 MB - 1 GB / 10-25 GB	from \$4/mo	Too small to build; runs prebuilt artifact with swap
s-1vcpu-1gb	1 / 1 GB / 25 GB	\$6/mo	Budget pick; add 2 GB swap for build safety
s-2vcpu-2gb	2 / 2 GB / 60 GB	\$18/mo	Workable; tight during next build
s-2vcpu-4gb	2 / 4 GB / 80 GB	\$24/mo (\$0.0357/hr)	Recommended; build and run on one box
Premium AMD/Intel	2 / 4 GB and up	about 15-30% over Basic	Faster clocks and NVMe for the same footprint

Table 4: Basic Droplet tiers relevant to this app (DigitalOcean pricing, verified Aug-Sep 2026).

3.1 App Platform versus Droplets

App Platform is DigitalOcean's managed PaaS: connect the GitHub repo, it builds and runs the container, handles TLS, and restarts on failure. Basic web services start at \$5 per month, static sites are free, dedicated instances scale up to \$156 per month, and each component bills separately, so a web service plus a background worker starts at \$10 per month. The honest catch for this codebase is persistence: App Platform containers have ephemeral filesystems, so the SQLite file would be wiped on every deploy. Adopting App Platform therefore implies migrating Prisma from the sqlite provider to Managed PostgreSQL, which is a schema-provider change plus a data migration, not just a connection-string swap. For teams that never want to touch a server, that migration is worth it; for a single-server operator, a \$24 Droplet with snapshots delivers the same reliability for less money and zero code changes.

3.2 Managed PostgreSQL and DOKS

Managed PostgreSQL starts at \$15 per month for a single node with 1 vCPU and 1 GiB of RAM, with automatic daily backups and encrypted at rest; high-availability two-node clusters start at \$30 per month. It is

the right upgrade when the platform accumulates real operational history that cannot be re-scraped, or when you want point-in-time recovery instead of snapshot-based file backups. Managed Kubernetes (DOKS) starts around \$12 per month for the control plane alone before worker nodes, and given that the whole application is one process plus cron-style jobs, DOKS adds operational surface without adding capability. Revisit it only if the platform becomes multi-tenant with isolated customer workloads.

Verdict: a Basic 2 vCPU / 4 GB Droplet at \$24 per month running the standalone server with SQLite, Caddy for TLS, and nightly file backups is the recommended configuration. Treat Managed PostgreSQL as a paid upgrade path, App Platform as the zero-ops alternative, and DOKS as not applicable today.

4. DigitalOcean GPU Droplets: The Mining Angle

DigitalOcean matters to this platform twice: once as the host for the control plane, and once as a provisioning provider that DevOps Engine can drive directly. The provider-key dialog in the application stores DigitalOcean credentials in the platform database (alongside RunPod, Vast.ai and Hetzner), after which the engine can create GPU Droplets through the DigitalOcean API exactly as it does for the other providers. That makes DigitalOcean the only mainstream cloud in the integration set where the web app and the miner fleet can live under one bill. The verified GPU catalog and rates follow; all of them bill per second with a 60-second minimum, so short experiments are cheap to abandon.

GPU (per-GPU hourly)	VRAM	Price	Availability notes
NVIDIA RTX 4000 Ada	20 GB	\$0.76/hr	Single-node SKU, Toronto (TOR1)
NVIDIA RTX 6000 Ada	48 GB	\$1.57/hr	Single-node SKU, TOR1
NVIDIA L40S	48 GB	\$1.57/hr	Single-node SKU, TOR1
NVIDIA H100 80 GB (SXM)	80 GB	from about \$3.39/hr	Multi-GPU nodes; 8x node \$35.28/hr (about \$4.41/GPU)
NVIDIA H200 / AMD MI300X	141 GB / 192 GB	quote/limited	Listed in the portfolio; availability varies by region

Table 5: DigitalOcean GPU Droplet catalog with on-demand rates (docs verified August 2026).

4.1 The honest mining economics

Cloud GPU rates and Bittensor subnet emissions do not mix well at the high end. An H100 at \$3.39-\$4.41 per hour costs roughly \$2,440-\$3,175 per month if run 24/7, which exceeds the emissions most subnets distribute to a single miner, so an H100 lease is a bet on a specific subnet's exceptional economics rather than a default. At the other end, the RTX 4000 Ada at \$0.76 per hour (about \$554 per month at full uptime) is the only DigitalOcean GPU whose rate sits in the same class as the consumer cards that typically mine, and it is

the right choice for a controlled trial. Before any long-running lease, check the platform's economic panels for the target subnet, set a stop-loss reminder for the first 48 hours, and remember that RunPod's community-cloud RTX 4090 at \$0.34-\$0.69 per hour undercuts DigitalOcean for pure experiments. DigitalOcean's advantages are the integrated API, predictable datacenter hardware, and per-second billing, not absolute price.

One operational note: single-GPU SKUs are concentrated in the Toronto (TOR1) datacenter, so provisioning code should treat region as a first-class parameter and surface honest errors when a region is out of stock instead of retrying silently. DevOps Engine's gate model (AUTO/MANUAL/APPROVAL) exists precisely for this: an out-of-stock or mispriced SKU should stop at an approval gate rather than drain the account.

5. Cross-Provider Comparison

DigitalOcean should still be benchmarked against the alternatives, both because the platform already integrates several providers and because 2026 pricing shifted. Hetzner raised cloud prices in June 2026 (some lines moved multiple-fold; the CPX11 went from 3.85 to 5.49 euros per month), which narrowed but did not close its raw performance-per-dollar lead over DigitalOcean. For the web app, the meaningful comparison columns are entry price, RAM at that price, operational effort, and whether the provider is integrated into the platform's provisioning engine.

Provider	Entry price	What you get	Verdict for this app
DigitalOcean	\$6-\$24/mo	1-4 GB RAM, 12 regions, GPU catalog, App Platform, integrated API	Best overall; one bill for control plane + provider API
Hetzner	EUR 5.49+/mo (CPX11)	2 GB RAM class, EU + some US regions, best raw CPU per euro	Best budget CPU after June 2026 hikes; not the integrated focus
Vultr	\$6/mo	1 GB RAM, 32 regions, similar catalog to DO	Fine DO substitute; no platform integration
Linode (Akamai)	\$5/mo	1 GB RAM, strong enterprise network	Legacy-stable, fewer modern extras
Railway	\$5/mo Hobby (usage)	PaaS, no server admin, volumes for SQLite	Good zero-ops alternative; usage pricing can surprise
Render	\$7/mo per service	PaaS, per-service pricing, auto-deploy	Web + worker model doubles cost; SQLite needs volumes

Table 6: Web-app hosting candidates, September 2026.

5.1 GPU providers versus DigitalOcean

Provider	RTX 4090 class	A100 / L40S class	H100 class	Integrated
DigitalOcean	n/a	L40S \$1.57/hr	\$3.39-\$4.41/hr	Yes
RunPod	\$0.34-\$0.69/hr	A100 \$1.39/hr	\$2.89-\$2.99/hr	Yes
Vast.ai	about \$0.30-\$0.40/hr	marketplace	marketplace	Yes

Table 7: GPU rates for the providers integrated by the platform (on-demand, verified Aug-Sep 2026).

Reading the two tables together produces a clean division of labor. DigitalOcean hosts the control plane and is the integrated provider of record when you want cloud-grade reliability on miner nodes; RunPod and Vast.ai win the metered-GPU experiment lane on price, especially for consumer cards; Hetzner is the value pick if you later colocate dedicated CPU miners and accept EU-centric regions. Nothing prevents running all of them at once, since the platform stores per-provider keys and routes each deployment to whichever provider you select. The only hard rule is to keep the control plane off GPU billing entirely, so a runaway mining experiment can never take the dashboard down with it.

6. Production Blueprint: Deploy on a DigitalOcean Droplet

This chapter is the copy-paste path from an empty account to HTTPS on a domain. It assumes the recommended 2 vCPU / 4 GB Basic Droplet on Ubuntu 24.04 LTS, a domain you control, and about thirty minutes. Every command runs as a non-root sudo user except where noted, and the final state is: systemd supervising the standalone server, Caddy terminating TLS, nightly SQLite backups, and the DigitalOcean monitoring agent reporting metrics.

6.1 Steps 1-4: create, harden, install, configure

- **Create the Droplet:** Ubuntu 24.04 LTS, Basic 2 vCPU / 4 GB, your SSH key injected, DigitalOcean Monitoring enabled, weekly snapshot policy on.
- **Harden it:** create a sudo user, disable SSH password auth, then UFW allow 22, 80, 443.
- **Install the runtime:** Bun (which the start script uses) plus git; clone the repository to /opt/infranex.
- **Configure secrets:** write /opt/infranex/.env with the five variables below; keep DEVOPS_SECRET stable across reinstalls so already-paired miners keep verifying.

```
Environment file
# /opt/infranex/.env (chmod 600)
DATABASE_URL=file:/opt/infranex/db/custom.db
APP_SESSION_SECRET=<64-hex>
DEVOPS_SECRET=<64-hex>
INFRANEX_REPO_ROOT=/opt/infranex
```

6.2 Steps 5-7: build, supervise, serve over TLS

Build the standalone artifact, install it as a systemd unit, and put Caddy in front for automatic Let's Encrypt certificates. Caddy is preferred over Nginx here because it obtains and renews certificates with zero configuration beyond the domain line, and its config file is five lines long. The unit below deliberately runs the server on 127.0.0.1:3000 so nothing is exposed except through Caddy.

Build

```
cd /opt/infranex
bun install
bunx prisma generate && bunx prisma db push
bun run build # next build + standalone copy
```

systemd unit

```
# /etc/systemd/system/infranex.service
[Unit]
Description=InfraNexBT web platform
After=network-online.target
[Service]
WorkingDirectory=/opt/infranex
Environment=NODE_ENV=production
EnvironmentFile=/opt/infranex/.env
ExecStart=/home/deploy/.bun/bin/bun /opt/infranex/.next/standalone/server.js
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
```

Caddyfile (auto-HTTPS)

```
# /etc/caddy/Caddyfile
app.example.com {
    reverse_proxy 127.0.0.1:3000
}
```

6.3 Steps 8-10: backups, monitoring, updates

- **Backups:** nightly cron running `sqlite3 db/custom.db '.backup /var/backups/custom-$(date +%F).db'`, then `rclone` to DO Spaces or another offsite target; keep 14 days.
- **Health probe:** `curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:3000/login` must return 200 or 307; anything else pages you via the DO monitoring webhook.
- **Clock discipline:** enable `systemd-timesyncd`; the daemon HMAC channel tolerates plus-or-minus 5 minutes of skew and then rejects telemetry, so time drift shows up as miner silence.
- **Updates:** `git pull`, `bun install`, `rebuild`, `systemctl restart infranex`; test on a snapshot-restored clone first if a migration touched the schema.

Two final operational rules. Never co-locate miners on the control-plane Droplet: the whole point of the separation is that mining experiments can burn an account without ever taking the dashboard down. And treat `APP_SESSION_SECRET` and `DEVOPS_SECRET` as environment-level state, not code: rotate them deliberately (sessions die on rotation, and miners must be re-paired if `DEVOPS_SECRET` changes), not as a side effect of a server rebuild. This incident already happened once in September 2026 when a sandbox reset silently dropped the variable from the environment file.

7. Cost Scenarios and FAQ

The three scenarios below bundle chapter 3 and 4 pricing into realistic monthly bills, followed by direct answers to the deployment questions that come up most often for this application. Prices are DigitalOcean list prices verified in September 2026; GPU spend is metered separately because it is decided per-experiment, not per-month.

Scenario	Components	Monthly
Minimal	1 vCPU / 1 GB Droplet (\$6) + snapshots (about \$1.20)	about \$8
Recommended	2 vCPU / 4 GB Droplet (\$24) + snapshots (\$5) + Spaces backup (\$5)	about \$34
Scale	Droplet (\$24) + Load Balancer (\$12) + Managed PG HA (\$30) + Spaces (\$5)	about \$71 + GPU

Table 8: All-in monthly cost scenarios (GPU fleet metered separately).

7.1 Frequently asked questions

Can App Platform run this app as-is? The web server deploys fine, but the SQLite file does not survive ephemeral container storage, and the seven background workers want a second always-on service. Practically that means App Platform plus Managed PostgreSQL plus a worker component: zero server admin for roughly \$35-\$40 per month, at the cost of a Prisma provider migration. If that migration does not appeal, the \$24 Droplet is the better buy.

Do I need Kubernetes or a load balancer? No. The application is a single stateful process with an in-database command queue; DOKS adds a control-plane fee and operational surface without adding capability. A Load Balancer earns its \$12 per month only when you run two app Droplets behind it for zero-downtime deploys, which is a scale-era concern.

Where do the RunPod, Vast.ai, Hetzner and DigitalOcean keys live? In the platform database, entered through the provider-keys dialog; only `RUNPOD_API_KEY` is also read from the environment as a fallback. That means migrating hosts means moving the database file (or the managed database), not just the `.env` file, and it is also why the database backup policy in chapter 6 matters beyond the web data itself.

Can the same server host miners? Technically the Droplet could run CPU miners, but it should not: mining workloads distort the control plane's latency, and the daemon telemetry the platform relies on would be measuring itself. Keep the dashboard on its own small box and provision miner hosts through DevOps Engine so gates, monitoring, and verdict push stay meaningful.

How does hosting interact with the daemon verdict channel? Miners pull commands from `/api/daemon/commands` and post signed telemetry every 60 seconds, so the control plane needs a stable public hostname with valid TLS (Caddy provides it), an accurate clock, and the shared `DEVOPS_SECRET`. Silence longer than 10 minutes raises a warning in the platform's monitor, which is the signal to check DNS, certificates, or server time first.

What if I outgrow SQLite? Move to Managed PostgreSQL deliberately: switch the Prisma datasource provider, run db push against the managed cluster, migrate the snapshot-exported data, and keep SQLite as a local dev convenience. The \$15 single node is enough for years of the platform's write volume, and the HA tier exists if the dashboard becomes revenue-critical.